

Data Types, Operators, and Conversion

Lecture 4

CS 1001

Kirk Duran

Today

- ▶ Why computer systems need data types.
- ▶ How Python represents integers, floating-point numbers, strings, and Boolean values.
- ▶ How operand types determine an operator's meaning and result type.
- ▶ How Python handles compatible and incompatible type combinations.
- ▶ How explicit conversion produces a value of another type.

Key idea

Every example remains a self-contained expression; naming and retaining values comes later.

Part 1: Why Data Types Exist

A Group of Bits Does Not Explain Its Own Meaning

One possible representation of “Kirk”

01001011 01101001
01110010 01101011

- ▶ At the hardware level, text and numbers are represented using bits.
- ▶ The bit pattern alone does not say whether it represents text, a number, an image, or instructions.
- ▶ Adding the number 5 to encoded text has no useful textual meaning.

Python int to Binary

Convert integers with bin(), format(), and f-strings



Key idea

A representation becomes a meaningful value only when a system knows how to interpret it.

Different Kinds of Values Need Different Interpretations

- ▶ **Whole numbers:** 42, -8
- ▶ **Fractional values:** 3.14, -0.25
- ▶ **Text:** "Cal Poly"
- ▶ **Two logical states:** True, False

basic data types



Example

Python names these types `int`, `float`, `str`, and `bool`.

Every Python Value Has a Type

Definition

A **data type** is a classification that determines how a value is represented, interpreted, and used.

Representation

How Python organizes the information that constitutes the value.

Operations

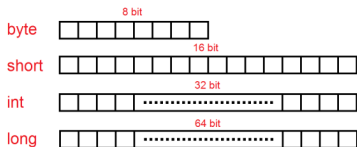
Which operators and functions are defined for the value.

Results

What value and type an operation produces.

Representations Introduce Costs and Limits

- ▶ A **fixed-size representation** uses a predetermined amount of storage.
- ▶ A **variable-size representation** can grow with the represented value.
- ▶ Numerical representations may trade exactness for range and efficiency.
- ▶ A value that does not fit must be shortened, rejected, approximated, or represented differently.



A Ten-Character Limit Shaped Feraligatr's Name

Ten characters fit

F E R A L I G A T R
1 2 3 4 5 6 7 8 9 10

Eleven characters do not

F E R A L I G A T O R

- ▶ Generation II limited English Pokémon names to ten characters.
- ▶ The shortened *Feraligatr* spelling fits that representation.



Key idea

A storage decision can become a visible design constraint.

Part 2: Fundamental Python Types

Integers Represent Whole Numbers Exactly

12

-4

0

1_000_000

- ▶ Python names this type `int`.
- ▶ Integer literals contain no decimal point.
- ▶ Integers represent positive whole numbers, negative whole numbers, and zero.
- ▶ Integer arithmetic can produce another integer.

Python Integers Grow as More Digits Are Required

- ▶ Many systems use fixed-width integers such as 8, 32, or 64 bits.
- ▶ Python integers use an arbitrary-precision representation.
- ▶ The representation grows to accommodate additional digits.
- ▶ The practical limit is available memory rather than a fixed 32-bit boundary.

Example

CPython implements integers using an internal structure named `PyLongObject`.

```
1 # Scientific Notation
2 x = 1e6
3 print(x) # 1000000.0
4
5
6 # PEP 515 - Underscores in Numeric Literals
7 y = 1_000_000.0
8 print(y) # 1000000.0 (float)
9
10 z = 1_000_000
11 print(z) # 1000000 (int)
```

Floats Represent Numerical Values with Finite Precision

3.5

-0.25

7.0

Example

7 and 7.0 describe the same numerical quantity but have different types.

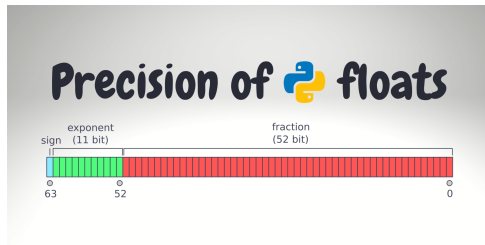
- ▶ Python names this type `float`.
- ▶ CPython commonly uses 64-bit IEEE 754 binary floating point.
- ▶ A float provides roughly 15–17 significant decimal digits.
- ▶ Its numerical range is large but finite.

The Point “Floats” by Changing the Scale

- ▶ The **sign** distinguishes positive and negative values.
- ▶ The **significand** contains the meaningful digits.
- ▶ The **exponent** determines the scale of the value.
- ▶ Changing the exponent effectively moves the binary point.

Key idea

Floating point supports both very small and very large magnitudes using a fixed amount of storage.



A Large Range Does Not Guarantee Exact Representation

- ▶ A finite number of bits can represent only finitely many values.
- ▶ Many decimal fractions do not have an exact finite binary representation.
- ▶ Python therefore stores the nearest representable floating-point value.

Example

0.1 + 0.2 may display
0.30000000000000004.

```
//round to the 10th decimal place (like actual 10th not 9th decimal)
//if the number is too high or low it ends as zero which is bad
System.out.println("Conditional pass: " + (answerNum<1.0E7 && answerNum>1.0E-7));
System.out.println("Conditional pass: " + (answerNum>1.0E7 && answerNum<1.0E-7));
//for fixing floating point rounding errors
if((answerNum<1.0E7 && answerNum>1.0E-7) || (answerNum>1.0E7 && answerNum<1.0E-7))
{
    answerNum=Math.round(answerNum/(Math.pow(10,(int)(Math.log10(Math.abs(answerNum))))+1))*Math.pow(10,(int)(Math.log10(Math.abs(answerNum))))+1);
}

//if double
if(Double.toString(answerNum).endsWith(".0"))
{
    return Integer.toString(Integer.parseInt(answerNum));
}
else
    return Double.toString(answerNum);
}
```

/*

- * Makes sure to get rid of any unwanted input from the text
- * **BASIC** equationEnter the textfield used to get the input & what is ultimately changed
- * **BASIC** equation determines what kind of input gets replaced

Strings Represent Unicode Text

`"hello"`

`"42"`

`"Cal Poly"`

`""`

- ▶ Python names this type `str`.
- ▶ A string is an immutable sequence of Unicode characters.
- ▶ Its representation grows with the amount and kind of text.
- ▶ Quotes delimit a string literal; digits inside quotes remain text.

Boolean Values Represent Two Possibilities

True

False

- ▶ Python names this type `bool`.
- ▶ The type contains exactly two literal values.
- ▶ Capitalization matters: `True` and `False`.
- ▶ Comparisons and Boolean operators will be introduced later.

`type()` Reports How Python Classifies a Value

Expression	Result
<code>type(7)</code>	<code><class 'int'></code>
<code>type(7.0)</code>	<code><class 'float'></code>
<code>type("7")</code>	<code><class 'str'></code>
<code>type(True)</code>	<code><class 'bool'></code>

Key idea

Python evaluates the argument first; `type()` then reports the resulting value's type.

Part 3: Operators Depend on Types

Operand Types Select the Applicable Operator Rule

Evaluation model

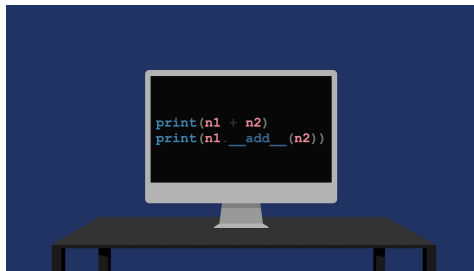
operand values and types



operator compatibility rule



result value and type



Key idea

An operator does not have one universal behavior independent of its operands.

Numeric Types Support Arithmetic Operations

Expression	Value	Type
<code>4 + 3</code>	7	int
<code>4.0 + 3.0</code>	7.0	float
<code>4 * 3</code>	12	int
<code>4 * 3.0</code>	12.0	float
<code>2 ** 3</code>	8	int

Example

The operator and operand types jointly determine the resulting value and type.

The Same + Symbol Can Mean Different Operations

Expression	Selected operation	Result
4 + 3	integer addition	7
4.0 + 3.0	floating-point addition	7.0
"4" + "3"	string concatenation	"43"

Strings Support Concatenation and Repetition

Concatenation

```
"Cal" + " Poly"
```

→

```
"Cal Poly"
```

Repetition

```
"ha" * 3
```

→

```
"hahaha"
```

Division Operators Can Produce Different Values and Types

Expression	Value	Type
<code>4 / 2</code>	2.0	float
<code>4 // 2</code>	2	int
<code>5 / 2</code>	2.5	float
<code>5 // 2</code>	2	int
<code>5 // 2.0</code>	2.0	float

Key idea

`/` always produces a float; floor division produces a float when a floating-point operand participates.

Compatible Mixed Numeric Types Produce a Numeric Result

Expression	Value	Type
<code>5 + 3.5</code>	8.5	float
<code>4 * 2.0</code>	8.0	float
<code>7 - 2.5</code>	4.5	float
<code>7 // 2.0</code>	3.0	float

Example

When an arithmetic expression combines an `int` and a `float`, Python generally produces a `float`.

Unsupported Type Combinations Produce a TypeError

- ▶ `4 + "3"`
- ▶ `"hello" - "h"`
- ▶ `"ha" * 2.5`
- ▶ `"4" / "2"`

Definition

A `TypeError` indicates that an operation received a kind of value it does not support.

Part 4: Conversion, Prediction, and Explanation

Explicit Conversion Requests a New Type

- ▶ `int("12") → 12`
- ▶ `float(8) → 8.0`
- ▶ `str(3.5) → "3.5"`

Key idea

The conversion expression produces a new value of the requested type.



Mastering Type Conversion In Python

Type Conversion In Real Life



Orange → Juicer → Juice

Type Conversion In Python

`"123" → int() → "123"`

String Method Integer

`int()` Produces an Integer Value

Expression	Value	Type
<code>int("12")</code>	12	<code>int</code>
<code>int(3.9)</code>	3	<code>int</code>
<code>int(8.0)</code>	8	<code>int</code>

Example

Converting 3.9 does not round to the nearest integer; `int()` removes the fractional portion toward zero.

float() and str() Produce Different Representations

Floating-point conversion

`float(8) → 8.0`

`float("3.5") → 3.5`

String conversion

`str(3) → "3"`

`str(3.5) → "3.5"`

Key idea

Conversion changes how Python interprets the resulting value; it does not merely change how the value is displayed.

Conversion Changes Which Operator Rule Applies

Text remains text

`"4" + "3"`

→

`"43"`

Convert before addition

`int("4") + int("3")`

→

`7`

Example

The conversions change the operand types, so Python selects numerical addition instead of string concatenation.

Explicit and Implicit Numeric Conversion Are Different

Conversion	What happens	Example
Explicit	The expression directly requests a target type.	<code>float(5)</code>
Implicit numeric	Python applies a compatible mixed-numeric rule.	<code>5 + 3.5</code>

Example

`5 + 3.5 → 8.5`

No guessing from text

`5 + "3.5" → TypeError`

Predict the Value and Type Before Running Python

1. `7 + 2`
2. `7 + 2.0`
3. `7 / 2`
4. `7 // 2`
5. `"7" + "2"`
6. `"ha" * 3`
7. `int("7") + 2`
8. `str(7) + "2"`

In class

For each expression, identify:

1. the operand types,
2. the selected operation,
3. the final value, and
4. the final type.

Use Colab to Inspect Both the Value and Its Type

1. Predict the value and type.
2. Run the expression in one cell.
3. Inspect the same expression using `type()`.
4. Compare the observations with the prediction.
5. Explain which type rule produced the result.

Example

```
7 + 2.0  
type(7 + 2.0)
```

Trace Types Through One Complete Expression

In class

Evaluate the expression without running Python first:

```
str(4 + 3.0) + " points"
```

1. `4 + 3.0` produces `7.0`, a float.
2. `str(7.0)` produces `"7.0"`, a str.
3. String concatenation produces `"7.0 points"`.

Key idea

To explain an expression, track both the value and the type produced by each subexpression.